PROJET CAPSTONE

Éthique & Sécurité

Mécanismes de sécurité

Authentification, cloisonnement, chiffrement, audit et réponse à incident — PrediCare TPA

Réalisé par :

ANDRIAMBOLOLONA Riantsoa Cherica
OLIVETTE Jason Brad Mirado
ANDRIAMIHAJA Henintsoa Jerry

Date : Mercredi 21 Octobre 2026

Périmètre	API cœur, scoring, assistant documentaire, interface web, BI, base, Redis, sauvegardes
Référentiels	STRIDE, OWASP ASVS, OWASP LLM Top 10, ISO/IEC 27001, NIST CSF 2.0

1. Synthèse

La plateforme est multi-rôle et manipule des données de santé pseudonymisées, des scores de fraude et des décisions financières. Sa sécurité repose sur cinq principes :

- cloisonner : chaque rôle ne voit que son périmètre, et le filtre est appliqué côté serveur ;
- minimiser : les noms sont chiffrés et isolés, les factures ne portent qu'un pseudonyme ;
- tracer : les connexions, décisions, changements de seuil, actions d'administration et actions RGPD sont journalisés ;
- chiffrer : les identités sont chiffrées et les sauvegardes le sont quand la clé est fournie ; TLS est disponible mais doit être activé en production ;
- limiter l'impact : base et Redis non publiés, SQL en lecture seule, limitation de débit, cache cloisonné.

Les contrôles démontrés sont réels : RBAC, Argon2id, JWT, filtres serveur, audit, chiffrement des identités, sauvegarde chiffrée, test d'intrusion interne, tests d'attaque de l'assistant, du SQL et du RBAC, règles d'alerte Prometheus et deux vues Grafana versionnées.

Les écarts principaux sont connus :

- TLS n'est pas activé par défaut ;
- MFA absent ;
- pas de quota de modèle par utilisateur ;
- pas de test d'intrusion externe ;
- le plan de surveillance après commercialisation n'est pas écrit, et la dérive (PSI) n'est pas encore exposée en métrique ;
- pas de test de restauration automatisé ;
- rotation des clés outillée mais peu testée ;
- état des dépendances et Dependabot à vérifier ;
- pas de SBOM ;
- procédure d'incident et de notification à formaliser.

La posture est donc solide pour une démonstration, partielle pour une production. Le document distingue les contrôles, les preuves et les risques résiduels.

2. Périmètre et actifs

2.1. Actifs

Actif	Sensibilité	Localisation
Identités chiffrées	Très élevée	PostgreSQL, table core.identite_chiffree
Factures et lignes	Élevée	PostgreSQL
Prédictions et explications	Élevée	PostgreSQL, ml
Journal d'audit	Élevée	PostgreSQL, audit
Sessions de l'assistant	Élevée	Redis
Clés et secrets	Critique	.env, secrets Docker
Sauvegardes	Critique	Volume backups
Fichiers de factures importés	Élevée	Volume temporaire d'import
Modèles	Moyenne	Volume models
Tableaux de bord BI	Moyenne	Métadonnées BI
Logs	Moyenne	Docker, Prometheus

2.2. Acteurs

Acteur	Accès
Assuré	Ses factures, son assistant, son export
Professionnel de santé	Ses dépôts, ses dossiers, ses ordonnances
Gestionnaire	File d'alertes, détail, décision
Administrateur	Comptes, seuil, modèle, RGPD, observabilité
Exploitant	Conteneurs, logs, sauvegardes, métriques
Fournisseur de modèle	Question, extraits, historique minimisé
Attaquant externe	Surface HTTP, BI, API

2.3. Frontières de confiance

Internet → Passerelle (HTTP/HTTPS, limitation de débit, en-têtes) → web / api-core / api-inference / api-rag / BI / automatisation → Réseau interne : PostgreSQL, Redis, volumes.

Les services internes ne doivent pas être exposés directement. Le navigateur ne doit jamais recevoir un secret de service.

3. Modèle de menace STRIDE

Menace	Scénario	Contrôle	Écart
Usurpation	Vol de jeton, force brute	JWT signé, Argon2id, mot de passe 12 car., rate limit	Pas de MFA

Altération	Injection SQL, injection de prompt	SQL lecture seule, liste blanche, séparation	Filtre non exhaustif
Répudiation	Nier une décision	Audit décisions, audit connexion, logs JSON	Actions admin à compléter
Divulgation	Fuite de noms, cache partagé	Chiffrement, pseudonymes, scope assuré	Logs à revoir
Déni de service	Script sur l'assistant	Rate limit, question 1 000 car., LIMIT SQL	Pas de quota par utilisateur
Élévation de privilèges	Admin via API	RBAC, permissions dédiées	MFA admin absent

3.1. Scénarios prioritaires

1. Vol d'un jeton de gestionnaire : l'attaquant peut consulter la file et décider. Mitigation : expiration courte, TLS, MFA, révocation, journalisation.

2. Injection de prompt : l'attaquant demande à l'assistant d'ignorer les instructions ou d'exfiltrer un prompt. Mitigation : filtre, séparation, pas d'outil d'écriture, SQL lecture seule.

3. Injection SQL via l'assistant : l'attaquant tente une requête destructive. Mitigation : compte predicare_ro, transaction en lecture seule, liste blanche, paramètres liés.

4. Fuite de sauvegarde : un volume est copié. Mitigation : chiffrement AES-256, clé séparée, accès restreint.

5. Accès indu d'un professionnel : un professionnel voit les factures d'un autre. Mitigation : filtre prestataire_id côté serveur, corrigé et testé.

6. Compromission d'un compte admin : l'attaquant change le seuil ou les secrets. Mitigation : MFA, moindre privilège, audit, rotation.

7. Déni de service sur le modèle : un utilisateur épuise le quota. Mitigation : limitation de débit, quota par sujet, alerte de coût.

8. Fuite par un fichier importé : une facture PDF contient l'identité du patient en clair. Mitigation : fichiers non journalisés, volume d'import temporaire, validation du type, purge après traitement.

4. Authentification et habilitations

4.1. Authentification

Contrôle	Implémentation
Hachage	Argon2id
Longueur minimale	12 caractères
Jeton	JWT signé, expiration, rôle, assure_id, session
Erreur de connexion	Message identique pour e-mail inconnu et mot de passe faux
Limitation de débit	60 requêtes/min, burst 120

Audit	Action connexion dans audit.journal
MFA	Absent

Le message d'erreur identique évite l'énumération des comptes. La connexion est journalisée dans la même transaction que la mise à jour de last_login : si l'écriture réussit, la trace existe.

4.2. Rôles

Rôle	Périmètre	Interdictions
Assuré	Ses factures, son export, son assistant	File d'alertes, admin, autres dossiers
Professionnel de santé	Ses dépôts, ses dossiers, ses ordonnances	File des autres praticiens, admin
Gestionnaire	File d'alertes, décision, explications	Administration, secrets
Administrateur	Comptes, seuil, modèle, RGPD, observabilité	Aucune, mais actions tracées

Le contrôle d'accès est centralisé dans packages/predicare-core/src/predicare/rbac.py et appliqué par des dépendances FastAPI. Une route non annotée est refusée par défaut dans les tests RBAC.

4.3. Cloisonnement des données

Donnée	Filtre	Preuve
Factures d'un assuré	facture.assure_id = assure_id du jeton	billing/router.py
Factures d'un prestataire	facture.prestataire_id = prestataire du compte	billing/router.py
Export RGPD	Pseudonyme du jeton ou admin	rgpd/router.py
Cache de l'assistant	assure_id ou user:{subject}	semantic_cache.py
SQL de l'assistant	Compte lecture seule + paramètres	sql_tool.py
Tableau de bord	Jeton d'invité, rôle invité	ops/superset/preparer_embed.py

Le filtre prestataire est un correctif important : auparavant, un professionnel pouvait voir toutes les factures pseudonymisées. Le code vérifie maintenant le prestataire du compte et l'applique dans la requête SQL. La soumission est également restreinte au prestataire du compte.

Une limite assumée : il n'y a pas de « Row Level Security » PostgreSQL. L'isolation entre assurés est portée par l'API et par l'outil SQL, pas par le moteur de la base. Tant qu'aucune autre application n'accède directement à la base, cela tient ; si c'est le cas, il faut activer le RLS.

5. Chiffrement et gestion des clés

5.1. Chiffrement au repos

- Les identités sont chiffrées avec Fernet (AES-128-CBC + HMAC).
- Le pseudonyme est un HMAC-SHA256 avec APP_SECRET_KEY.
- Les sauvegardes sont chiffrées avec AES-256-CBC, PBKDF2 200 000 itérations, clé lue depuis un fichier, lorsque BACKUP_CLE_FILE est présent ; sinon elles restent en clair et le journal le signale.
- Les secrets ne sont pas versionnés ; .env est ignoré.
- Les factures et prédictions ne contiennent pas de noms.

5.2. Chiffrement en transit

- En local, HTTP sur le réseau Docker.
- TLS est disponible via scripts/generer-traefik.sh : si PREDICARE_TLS_DOMAIN est défini et si les certificats sont fournis, la passerelle expose 443 et redirige HTTP.
- TLS n'est pas activé par défaut, car un certificat auto-signé sur localhost dégrade l'expérience et donne une fausse impression de sécurité.
- En production, TLS doit être obligatoire, avec certificats valides et HSTS.

5.3. Gestion des clés

Clé	Usage	Rotation
APP_SECRET_KEY	Pseudonymisation, signature interne	scripts/rotater-cles.sh
JWT_SECRET_KEY	Signature des jetons	scripts/rotater-cles.sh
IDENTITY_ENCRYPTION_KEY	Chiffrement Fernet	scripts/rotater-cles.sh
BACKUP_CLE_FILE	Chiffrement des sauvegardes	Procédure manuelle
LLM_API_KEY	Appels au modèle	Fournisseur
EMBEDDING_API_KEY	Vecteurs	Fournisseur
PREDICARE_DB_RO_URL	Lecture SQL de l'assistant	ALTER ROLE

La rotation des clés applicatives est outillée : le script simule par défaut, puis rechiffre les identités et les secrets dans une transaction avant de réécrire .env. Il impose un redémarrage. La rotation de la clé de sauvegarde n'est pas automatisée : une sauvegarde chiffrée avec une clé perdue est une sauvegarde perdue.

6. Réseau et passerelle

Contrôle	État
PostgreSQL non publié sur l'hôte	Démontré

Redis non publié sur l'hôte	Démontré
Passerelle sans socket Docker	Démontré
En-têtes CSP, nosniff, frame-deny	Configurés
HSTS	À activer avec TLS
Limitation de débit	60/min, burst 120
Journal d'accès	Activé
BI accessible directement	Oui en local ; à restreindre en production
Boîte mail de développement	Développement uniquement

Le réseau Docker isole les services internes. La passerelle est le seul point d'entrée. En production, le BI et les outils d'observabilité doivent être protégés par authentification, VPN ou liste d'accès.

7. Sécurité applicative et sécurité de l'assistant

7.1. Contrôles

Contrôle	Détail
Validation d'entrée	Schémas de validation, bornes, types
SQL	Compte predicare_ro, transaction lecture seule, liste blanche, paramètres liés
Injection de prompt	Motifs déclarés, refus explicite, journalisation
Séparation	Question dans un bloc <question>, instructions système séparées
Longueur	Question limitée à 1 000 caractères, historique borné
Périmètre	assure_id du jeton, jamais le texte de la question
Cache	Cloisonné par assuré, pas de cache pour les questions de suivi
Réponse	Sources citées, extraits, scores
Écriture	Aucun outil d'écriture pour l'agent
Explication	shap_available signale une explication indisponible
Information	Mention « Réponse générée par un assistant IA » affichée

7.2. Import de factures

L'import accepte aujourd'hui des fichiers PDF (une facture par fichier), des JSON et des CSV. C'est une nouvelle surface d'attaque, et c'est le seul endroit où l'identité d'un patient apparaît en clair.

Contrôle	Ce qui est fait
Type de fichier	Un lot PDF n'accepte que des .pdf ; limite de 20 fichiers par lot contrôlée
Isolation	Chaque facture est traitée séparément : un PDF illisible produit un échec isolé

Erreurs explicites	Un PDF sans couche de texte renvoie « PDF sans couche texte : OCR requis »
Journalisation	L'événement d'import est journalisé, pas le contenu du fichier
Vie du fichier	Le volume d'import est temporaire ; il doit être purgé après traitement

Point de vigilance : la minimisation s'arrête ici. Tout ce que la plateforme fait ailleurs — pseudonymes, chiffrement, cloisonnement — n'a pas d'équivalent sur le fichier importé. Le fichier est donc la donnée la plus sensible en circulation, et le lieu où un contrôle de suppression après traitement doit être écrit.

7.3. Limites

Le filtre d'injection est fondé sur des motifs : il ne couvre pas toutes les formulations. La barrière qui ne dépend pas du vocabulaire est le compte SQL en lecture seule et l'absence d'outil d'écriture. Le périmètre de lecture est tenu par l'injection du assure_id du jeton et la liste blanche, pas par le rôle SQL.

Le cache sémantique peut servir une réponse à une question proche. Le seuil de similarité (0,95 par défaut) doit être surveillé : trop bas, il mélange des questions ; trop haut, il ne sert à rien. Une réponse de cache ne doit jamais franchir un périmètre.

Côté explicabilité, l'API ne renvoie plus de contributions « à zéro » quand l'explication n'a pas pu être calculée : elle renvoie une liste vide et un drapeau shap_available = false, et le journal serveur trace l'incident. L'interface ne consomme pas encore ce drapeau : elle affiche « aucune contribution enregistrée », ce qui ne distingue pas « pas de contribution » de « explication indisponible ». C'est un petit écart à traiter.

8. Audit et supervision

8.1. Journalisation

Élément	Contenu	Preuve
Journal d'audit	Acteur, rôle, action, cible, détails, date	audit.journal
Décisions	Prédiction, décision, commentaire, acteur	audit.decision_gestionnaire
Connexion	Action connexion	identity/router.py
Seuil	Ancienne et nouvelle valeur	audit.parametre
Modèle	Version, artefact, métriques, statut	ml.modele_version
Dérive	PSI, variables, alerte	ml.rapport_derive
Comptes, canaux, secrets	Création, statut, configuration	audit.journal
RGPD	Export, rectification, opposition, effacement	audit.journal

Purge	Exécution planifiée, lignes supprimées par table	audit.journal

Les logs sont en JSON, sans noms. Les identifiants de facture et de prédiction restent des données pseudonymisées : ils doivent être traités comme des données personnelles.

8.2. Supervision

Prometheus collecte les métriques applicatives : latence, erreurs, tokens, coût, cache. Les règles d'alerte et les tableaux de bord sont désormais versionnés dans le dépôt : neuf règles dans ops/prometheus/rules/predicare.yml et deux vues Grafana (service et modèle) dans ops/grafana/provisioning/dashboards/.

Alerte	Ce qu'elle surveille
PredicareApiIndisponible	Une API ne répond plus
PredicareApiTauxErreur5xx	Pic d'erreurs serveur
PredicareApiLatenceP95	Latence p95 des API
PredicareInferenceErreurs	Erreurs de scoring
PredicareInferenceLatenceP95	Latence p95 de scoring
PredicareAucunePredictionRecente	Plus aucune facture notée
PredicareRagLatenceP95	Latence p95 de l'assistant
PredicareRagLlmIndisponible	Fournisseur de modèle injoignable
PredicarePartScoresEleves	Part anormalement élevée de scores hauts

Les signaux à surveiller restent : taux d'erreur 5xx, latence p95, tokens et coût par requête, taux de cache, dérive PSI, files d'alertes, espace disque, échecs de sauvegarde, connexions échouées.

Deux limites à connaître. Le PSI est calculé et persisté, mais pas encore exposé en métrique : l'alerte de dérive repose donc sur un indicateur de substitution (part des scores au-dessus de 0,3). Et le plan de surveillance après commercialisation — seuils d'escalade, responsable, périodicité — reste à écrire.

9. Sauvegarde, restauration et résilience

Élément	Valeur
Fréquence	Quotidienne, 03 h 00
Rétention	14 jours
Format	pg_dump compressé, chiffré si clé présente
Chiffrement	AES-256-CBC, PBKDF2 200 000 itérations
Vérification	gzip -t ou déchiffrement + gzip -t
Restauration	make restore, confirmation obligatoire
Sauvegarde avant restauration	Oui

RPO	≤ 24 h
RTO	Non mesuré
Test de restauration CI	Absent

La restauration n'est jamais automatique : une erreur de script écraserait les données du jour. Le script vérifie qu'au moins 10 tables existent après restauration, ce qui détecte une restauration vide.

La purge automatique des données hors délai, elle, ne touche pas aux sauvegardes : une archive peut conserver un dossier jusqu'à sa rotation, donc jusqu'à 14 jours après un effacement.

10. Chaîne d'approvisionnement

Contrôle	État
Images Docker avec tags explicites	Oui
Verrou Python uv.lock	Oui
CI GitHub Actions	Oui
Dépendances épinglées	Partiel
Dependabot	Aucun fichier de configuration ; état GitHub à vérifier
SBOM	Absent
Scan d'images	À vérifier
Signature d'images	Absent
Revue des dépendances	Partielle

Avant production, il faut activer Dependabot ou un équivalent, produire un SBOM, scanner les images et documenter la politique de mise à jour.

11. Preuves de test

Suite	Résultat	Commande
Tests unitaires et d'intégration	784 passés, 18 ignorés	make check
Attaques de l'assistant	97 tests, 100 % bloquées	pytest tests/test_rag_attacks.py
Garde-fous	78 tests	pytest tests/test_guardrails.py
SQL	57 tests	pytest tests/test_sql_tool.py
RBAC	22 tests	pytest tests/test_rbac.py
Chaîne d'automatisation	13/13 vérifications	make e2e-n8n
Import PDF	Extraction, validation, intégration	pytest tests/test_pdf_facture.py
Purge planifiée	Exécution, traçabilité, idempotence	pytest tests/test_purge_planifiee.py
Explications et parité	SHAP, parité des variables	pytest tests/test_explainability.py

Intrusion	7 familles de contrôles	scripts/test-intrusion.sh
Lint	OK	make lint
Types	OK	make typecheck

Les 18 tests ignorés sont les tests e2e et PostgreSQL sans pile démarrée ; ils sont rejoués sur la plateforme de démonstration. Le test d'intrusion est une liste de contrôles nommés, pas un scanner de vulnérabilités.

12. Réponse à incident

12.1. Étapes

1. Détecter : alerte, log, signalement, audit.
2. Qualifier : gravité, périmètre, données, personnes.
3. Confiner : révoquer les jetons, isoler le service, bloquer l'accès.
4. Préserver : logs, horodatage, captures, base.
5. Corriger : correctif, rotation, restauration.
6. Notifier : autorité sous 72 h si violation RGPD ; autorité AI Act si incident grave.
7. Apprendre : retour d'expérience, test de non-régression.

12.2. Incidents types

Incident	Première action	Vérification
Jeton volé	Révoquer, changer le secret, auditer	Sessions fermées
Fuite de clé de modèle	Révoquer chez le fournisseur, changer .env	Aucun appel non autorisé
Injection réussie	Bloquer l'utilisateur, analyser les logs	Test de non-régression
Facture PDF malveillante	Refuser le lot, journaliser sans le contenu	Aucun fichier conservé
Sauvegarde corrompue	Garder l'archive, tester la précédente	Restauration sur base vide
Disque plein	Purger les logs, réduire la rétention	Métriques stables
Base indisponible	Vérifier le conteneur, le volume, les logs	/health OK
Dérive du modèle	Revenir à la version précédente	Métriques et alertes
Accès indu	Couper le compte, auditer, corriger le filtre	Test RBAC

13. Risques et feuille de route

13.1. Registre des risques

Risque	Prob.	Impact	Contrôle	Résiduel

Vol de jeton	Moyenne	Élevé	JWT, TLS, rate limit	Moyen
MFA absent	Moyenne	Élevé	À ajouter	Élevé
Fuite de sauvegarde	Faible	Élevé	Chiffrement	Faible
Perte de clé	Faible	Élevé	Sauvegarde de clé	Moyen
Injection de prompt	Moyenne	Moyen	Filtre, SQL lecture seule	Faible
Déni de service modèle	Moyenne	Moyen	Rate limit	Moyen
Dépendance vulnérable	Moyenne	Moyen	Dependabot à activer	Moyen
Surveillance incomplete	Moyenne	Moyen	Règles Grafana ; plan à écrire	Moyen
Fuite par fichier importé	Moyenne	Élevé	Type, volume temporaire, non journalisé	Moyen
Pas de RLS PostgreSQL	Faible	Élevé	Isolation API ; RLS si besoin	Moyen
Restauration non testée	Moyenne	Élevé	Test à automatiser	Élevé
TLS non activé	Élevée	Élevé	Activer en production	Élevé

13.2. Priorités

Priorité	Action	Échéance
P0	Activer TLS en production	Avant production
P0	Ajouter MFA pour admin et gestionnaire	Avant production
P0	Tester une restauration complète	Avant production
P0	Écrire la procédure de purge des fichiers importés	Avant production
P1	Écrire le plan de surveillance et exposer le PSI	Avant production
P1	Activer Dependabot et scanner les images	Avant production
P1	Écrire la procédure d'incident 72 h	Avant production
P1	Ajouter un quota de modèle par utilisateur	Avant production
P1	Distinguer « aucune contribution » et « explication indisponible »	Avant production
P2	Activer le RLS PostgreSQL si besoin	Après production
P2	Produire un SBOM	Après production
P2	Test d'intrusion externe	Après production
P2	Rotation de la clé de sauvegarde	Après production

14. Conclusion

La sécurité de la plateforme est démontrée sur les contrôles structurants : RBAC, filtres serveur, chiffrement, audit, SQL en lecture seule, sauvegarde chiffrée, règles d'alerte,

tests. Elle reste partielle pour la production : TLS, MFA, test de restauration, gestion des dépendances, plan de surveillance et procédure d'incident.

La priorité n'est pas d'ajouter des outils, mais de fermer les écarts qui ont un impact direct sur la confidentialité, l'intégrité et la disponibilité. TLS et MFA protègent les comptes ; la restauration testée protège les données ; le plan de surveillance protège la disponibilité ; Dependabot protège la chaîne d'approvisionnement.